Rotaract Club of Xiamen University Malaysia

Rotary Club of Kuala Lumpur Dynamics

Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick “No” or write “None” – do not leave it blank. Submit as a single PDF document named “SDC2026_TeamName_TechDeclaration.pdf” together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

1. TEAM AND PROJECT OVERVIEW

Team Name

Bashamel

Project / Solution Name

Smart Inventory & Supplier Order Management System

Name (Leader)

Tatiana Razak

Email & Phone (Leader)

tatyrazak@gmail.com (+60 192204467)

University Affiliation(s)

Asia Pacific University

Live Prototype URL

https://smart-inventory-sdc.replit.app

Code Repository URL (public / access granted to judges)

https://github.com/ken2313/SDC-HACKATHON

2. ARCHITECTURE & TECHNOLOGY STACK

2.1. System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

Browser to backend: the React 19 interface (Vite) is hosted on Replit as the single public URL and sends /api requests to the FastAPI backend. The backend signs users in, enforces owner and employee permissions, applies the business rules (stock movements, low-stock alerts, draft purchase orders, supplier order tracking, demand forecasting) and stores everything in SQLite, including an append-only activity log. Orbyte is simulated inside the backend; no official Orbyte API is called.

Backend to AI model: after a stock-out is saved, a background task sends that day's usage figures (SKU, quantity, day of week, recent average and standard deviation) over HTTPS, with a secret token and a RunPod API key, to our own inference service on a RunPod Serverless GPU endpoint (a Docker container). It runs a self-trained PyTorch autoencoder on CUDA and returns Normal, Unusual or Highly Unusual. The backend stores the result as a flag for human review.

The call runs in the background with a long timeout (90 seconds, for cold starts) and one retry, and the app works normally if the AI is unavailable. Secrets are kept in Replit Secrets. No generative-AI API and no external database are used.

2. ARCHITECTURE & TECHNOLOGY STACK

2.2. Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.11	Language for the backend, tests and the AI service	PSF License (open source)
FastAPI 0.110+, Uvicorn 0.29+	Web framework and server for the backend API and for the AI inference service	MIT (FastAPI); BSD-3-Clause (Uvicorn)
SQLAlchemy 2.0+, Pydantic 2.6+, SQLite	Database access to a SQLite file, and validation of inputs and outputs	MIT (SQLAlchemy, Pydantic); SQLite is public domain
PyTorch 2.5.1 (Docker image), NumPy, pandas	Training and running the anomaly-detection autoencoder; preparing its input features	BSD-3-Clause (PyTorch, NumPy, pandas)
React 19, React Router 7, Recharts 3 (JavaScript)	Web interface, page routing and charts	MIT
Vite 7, Tailwind CSS 4; pytest, httpx, python-dotenv	Build tooling and styling; automated tests and configuration	MIT (Vite, Tailwind CSS, pytest); BSD-3-Clause (httpx, python-dotenv)

3. APIs & CLOUD SERVICES

3.1. External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API and Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
RunPod Serverless GPU endpoint: our own inference API (FastAPI /score) in a Docker container	The backend sends each stock-out's usage figures and receives an anomaly status (Normal, Unusual, Highly Unusual) for human review. Protected by a secret token and a RunPod API key.	Paid (RunPod Serverless, billed per second of GPU use)
Orbyte platform APIs (Alpha Red Solutions)	Not called. Orbyte is simulated inside our backend (/api/orbyte-mock); these are not official Orbyte APIs.	None used
GenAI / LLM APIs (OpenAI, Gemini, Claude, etc.)	None. The application contains no language-model or GenAI API call.	None used
Maps, payments, messaging, data feeds	None.	None used

3. APIs & CLOUD SERVICES

3.2. Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit (hosting)	Runs the whole app (FastAPI backend and React frontend, via run.sh) at https://smart-inventory-sdc.replit.app. Secrets are in Replit Secrets.	Replit-managed hosting; region set by Replit
RunPod Serverless (RunPod)	Compute for the anomaly model: ai/server.py runs in a Docker container on a CUDA GPU worker and answers /ping and /score requests.	Serverless, pay per second of use; scales to zero when idle
GitHub (GitHub, Inc.)	Source-code hosting and teamwork (branches and merges). Replit pulls the code from it.	Public repository
Database: none in the cloud	All business data and the activity log are in a SQLite file on the Replit app's disk.	Local file, no separate service
Authentication and storage: none	Demo sign-in is checked by our own backend (salted hash, signed session). No third-party auth or storage service.	None used

4. AI TOOLS, MODELS & DATASETS

4.1. AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
7-day moving-average forecast	Own implementation (statistics, not ML)	Runs in the backend to estimate demand and run-out dates, with a backtested accuracy score.	Own work
Anomaly autoencoder, autoencoder-v1 (PyTorch, 5-3-5)	Our own team (Bashamel)	Self-trained from scratch on synthetic normal usage data; runs on a RunPod GPU. Flags unusual stock-outs for human review (95th and 99th percentile thresholds).	Own work; no third-party model weights
Pre-trained, fine-tuned or GenAI models	None	None used. No language model or external model is called.	Not applicable

4. AI TOOLS, MODELS & DATASETS

4.2. Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Public or third-party datasets	None	None used	Not applicable	N
Synthetic usage data for the anomaly model	Generated by ai/data_gen.py in our repository (5 products, 120 days)	Synthetic: normal usage for training, injected anomalies for testing	Own work	N
Demo business data (Kopi Kita Cafe, Kedai Runcit Pak Din)	Generated by backend/app/seed in our repository	Synthetic: products, suppliers, orders and about 30 days of stock history	Own work	N (invented names and example emails)

4. AI TOOLS, MODELS & DATASETS

4.3. AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☒ Yes ☐ No

If yes, list them below:

Tool & Version	Used For	Extent of Use (minor / moderate / major)
Claude Code (Anthropic), Sonnet 5.5	Generated the backend and frontend code (API, services, database, tests, React pages), directed and tested by the team.	Major (backend and frontend code)
Claude (Anthropic), Sonnet 5.5, via Claude Code	Drafting the README and the submission documents (preliminary proposal, this declaration), and reading the repository to summarise features and check claims.	Moderate (documentation)

4. AI TOOLS, MODELS & DATASETS

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

We used AI to build the app. Claude Code (Sonnet 5.5) generated the backend and frontend code. Replit was used only to host and publish the app, not to generate code. The team directed the work, ran the app against our written manual demo scripts, and ran the automated backend tests (over 200), fixing what failed. Documents drafted with AI help were checked against the repository and are reviewed by the team leader before submission. The team takes responsibility for all submitted work.

5. SPONSOR-PROVIDED RESOURCES

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☒ Yes ☐ No

If yes, list them below:

Sponsor	Resource (API / credits / platform / data)	How It Was Used in the Project
Replit	Hosting platform for the app	Runs the live demo at smart-inventory-sdc.replit.app
RunPod	Serverless GPU endpoint for AI inference, and the SDC workshop on deploying models on RunPod Serverless	Runs our PyTorch anomaly model in a Docker container on a CUDA GPU. We applied the workshop's approach: an HTTP endpoint with health checks.
SDC Phase 1 workshops	Online seminars (mentoring)	Cloud deployment and database architecture ideas applied in the project.

5. SPONSOR-PROVIDED RESOURCES

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

No, it was not essential to the core app. Inventory, alerts, ordering and forecasting run on SQLite and need no sponsor resource, and the AI check is optional: if the AI service is down, the app carries on and shows "AI unavailable". Replit could be replaced by any host that runs Python and Node. RunPod's GPU runs the anomaly model today, but the model is very small and could run on a CPU or another GPU host. The workshops saved us time and shaped our design.

6. SECURITY, PRIVACY & RESPONSIBLE USE

6.1. Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The app collects no real personal data. All business data (products, stock, suppliers, orders) and the demo users (invented names, example email addresses) are synthetic. Sign-in is one shared demo account; its password is stored only as a salted PBKDF2-SHA256 hash, and sessions are signed tokens that expire after 12 hours.

Data is stored in a SQLite file on the Replit app, separated per business, and kept until the demo is reset.

The only data sent to a third-party service is numeric usage figures (SKU, quantity, day of week, recent average, standard deviation, history length) sent to our own model service on RunPod over HTTPS with a secret token and a RunPod API key. No names, emails or free text are sent, and nothing goes to any GenAI API. Secrets are kept in Replit Secrets, not in the code.

6. SECURITY, PRIVACY & RESPONSIBLE USE

6.2. Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

The autoencoder is trained only on synthetic data, so it may flag normal busy days or miss real problems. On our test set it reached recall 0.91 and precision 0.56, and a simple z-score baseline scored as well or better, so we do not claim it beats statistics here. It scores only stock-outs, needs at least 7 days of history, and depends on one provider (RunPod), and the first check after idle is slow (cold start).

Mitigations: flags are labelled potential anomalies for human review and never change stock; the check runs in the background after the stock change, with a 90 second timeout for cold starts and one retry, and the app shows "AI unavailable" instead of failing; requests need a secret token and an API key. No generative model is used, so there is no hallucination risk. Other limits: SQLite only, one demo account, Orbyte simulated.

7. COMPLIANCE & READINESS

Tick all boxes that apply before submitting.

- [x] All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.

- [x] All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.

- [x] No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.

- [x] Any use of generative-AI tools for development has been disclosed and reviewed by the team.

- [x] Any use of official SDC sponsor resources has been declared in Section 5.

- [x] The prototype described here is accessible to judges through the links provided in Section 1.

- [x] All fields in this template have been completed within the word limits stated.

8. DECLARATION

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature]



[Date]

24/10/2026